TERA VISION

MISSION FORGE: BUILD YOUR WAY TO SPACE

Document d'architecture, de répartition des responsabilités et de développement

Projet : NASA Space Apps Challenge 2026
Défi : Space Mission Design Game
Équipe : TERA VISION
Nombre de développeurs : 5
Type d'application : Jeu web interactif
Langage principal : TypeScript

PARTIE I — COMPRENDRE LE PROJET AVANT DE LE DÉVELOPPER

1.1. Quelle est notre vision ?

MISSION FORGE est un jeu éducatif dans lequel les joueurs doivent concevoir et gérer une mission spatiale.

Ils ne doivent pas simplement construire une fusée et la lancer. Ils doivent réfléchir comme une véritable équipe d'ingénieurs spatiaux.

Ils devront notamment :

- Choisir une mission scientifique.
- Déterminer la destination.
- Construire un vaisseau spatial.
- Sélectionner les équipements nécessaires.
- Respecter un budget.
- Gérer la masse et la consommation d'énergie.
- Prévoir les moyens de communication.
- Faire face aux pannes et aux imprévus.
- Analyser les résultats de leur mission.

Le jeu doit également expliquer les conséquences des décisions prises par les joueurs.

Notre objectif pédagogique : faire comprendre qu'une mission spatiale est un ensemble de décisions liées entre elles. Une bonne décision dans un domaine peut provoquer des difficultés dans un autre.

Par exemple :

Un joueur choisit un équipement scientifique très performant. Cet équipement améliore les possibilités de recherche, mais augmente la masse du vaisseau, sa consommation énergétique et son coût.

Il faudra donc trouver un compromis.

1.2. Comment se déroule une partie ?

Nous proposons de diviser une partie en sept grandes étapes.

Étape	Ce que font les joueurs	Ce que fait le jeu
1. Briefing	Découvrent le défi scientifique	Présente les objectifs, les contraintes et le budget
2. Conception	Choisissent les équipements et construisent le vaisseau	Calcule les caractéristiques du vaisseau
3. Vérification	Vérifient leur conception	Détecte les problèmes et les incompatibilités
4. Défense du projet	Présentent et justifient leurs décisions	Évalue la cohérence du projet
5. Lancement	Démarrent la mission	Simule le déroulement du voyage
6. Imprévus	Réagissent aux événements	Déclenche des pannes ou des situations difficiles
7. Autopsie	Analysent les résultats	Explique les réussites, les échecs et leurs causes

L'autopsie n'est pas une simple page affichant un score.

C'est une analyse qui doit permettre de répondre à ces questions :

- Qu'est-ce qui a fonctionné ?
- Qu'est-ce qui a échoué ?
- Quelle décision a provoqué le problème ?
- Quelles conséquences cette décision a-t-elle entraînées ?
- Comment aurait-on pu éviter cet échec ?

1.3. Les autres modes du jeu

Le cahier des charges prévoit également trois possibilités.

A. Les missions historiques

Les joueurs pourront découvrir et rejouer des missions spatiales réellement réalisées.

Exemples :

- Apollo 11.
- Voyager 1.
- Mars Pathfinder.
- Perseverance.

Le jeu devra présenter le contexte historique, les objectifs et les caractéristiques documentées de ces missions.

B. Les missions de science-fiction

Les joueurs pourront participer à des scénarios imaginaires.

Exemples :

- Explorer une planète inconnue.
- Installer une station scientifique sur Mars.
- Rechercher des traces de vie extraterrestre.
- Envoyer une sonde dans un système planétaire lointain.

C. L'explorateur de l'univers

Une interface permettra de découvrir différents objets célestes et leurs caractéristiques.

Exemples :

- Planètes.
- Lunes.
- Étoiles.
- Systèmes planétaires.
- Corps célestes remarquables.

PARTIE II — ORGANISATION DES CINQ MEMBRES

2.1. La répartition proposée

Pour que le travail soit équilibré et que chacun ait une responsabilité claire, voici la répartition recommandée.

Membre	Responsable de	Nom du module
Membre 1	Interface générale et expérience des joueurs	Game Interface
Membre 2	Conception et construction des vaisseaux	Mission & Spacecraft Designer
Membre 3	Calculs scientifiques et simulation spatiale	Simulation Engine
Membre 4	Multijoueur, sauvegarde et données	Multiplayer & Data
Membre 5	Événements, missions historiques et autopsie	Mission Intelligence

Cette répartition ne signifie pas que chaque membre travaille isolément.

Chaque membre développe son module, mais doit respecter les règles communes définies dans ce document.

2.2. Les jeux de référence pour chaque membre

Chaque responsable devra étudier certains jeux afin de comprendre les mécaniques dont il peut s'inspirer.

Membre	Jeu principal de référence	Ce qu'il doit observer
1	Mars Horizon	Navigation, tableaux de bord et présentation des informations
2	Kerbal Space Program	Construction des fusées et choix des composants
3	Juno: New Origins	Calculs, performances et simulation
4	Mars Horizon	Gestion des équipes et progression des missions
5	Kerbal Space Program + Universe Sandbox	Événements, exploration et visualisation des conséquences

Un autre jeu, Spaceflight Simulator, servira de référence commune pour observer comment rendre la construction des engins simple et compréhensible.

Attention : nous ne cherchons pas à copier ces jeux. Nous voulons comprendre leurs bonnes idées et les adapter à notre projet.

PARTIE III — ARCHITECTURE GÉNÉRALE DU PROJET

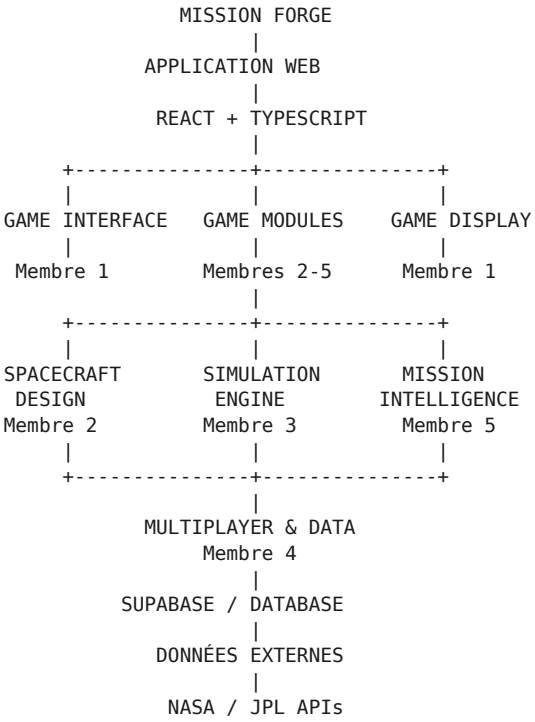
3.1. Comment les différentes parties vont-elles communiquer ?

Imaginez une entreprise qui construit un vaisseau spatial.

- Le premier membre construit les salles dans lesquelles les joueurs travaillent.
- Le deuxième conçoit le vaisseau.
- Le troisième vérifie si le vaisseau peut réellement accomplir la mission.
- Le quatrième permet aux joueurs de travailler ensemble et sauvegarde leur progression.
- Le cinquième organise les événements et explique les résultats.

Les cinq parties doivent communiquer entre elles.

Voici l'architecture générale :



Ce schéma représente les responsabilités fonctionnelles. Il ne signifie pas que toutes les communications doivent obligatoirement passer par le serveur.

Par exemple, les calculs de masse et de consommation peuvent être effectués directement dans le navigateur.

3.2. Les technologies communes à toute l'équipe

Nous devons éviter que chacun choisisse son propre langage ou ses propres outils.

Voici notre environnement commun.

Technologie	À quoi sert-elle ?	Pourquoi la choisir ?
TypeScript	Langage de programmation	Tout le monde utilise les mêmes règles de programmation
React	Création des interfaces	Permet de construire le jeu avec des éléments réutilisables

Technologie	À quoi sert-elle ?	Pourquoi la choisir ?
Vite	Démarrage et compilation du projet	Facilite le développement
Tailwind CSS	Mise en forme des pages	Permet de créer rapidement une interface propre
Zustand	Partage des informations entre les modules	Permet à toutes les parties d'accéder aux mêmes données du jeu
Git	Gestion des modifications du code	Évite de perdre le travail des autres
GitHub	Partage du code	Permet de travailler à cinq sur le même projet
Vitest	Vérification des calculs et fonctions	Permet de détecter les erreurs
Supabase	Sauvegarde des données et gestion des joueurs	Évite de développer tout un système de stockage et d'authentification

Pourquoi TypeScript ?

TypeScript est un langage proche de JavaScript, mais qui permet de mieux contrôler les types de données.

Par exemple, un budget doit être un nombre et non un texte.

Cela permet de réduire certaines erreurs et facilite le travail entre les membres.

Pourquoi Zustand ?

Imaginez que le membre 2 ajoute un moteur à un vaisseau.

Le membre 3 doit immédiatement pouvoir récupérer les informations de ce moteur pour effectuer ses calculs.

Zustand permet de partager les informations importantes entre les différentes interfaces du jeu.

Pourquoi ne pas utiliser immédiatement un moteur 3D ?

Parce que notre priorité est de construire un jeu fonctionnel.

Nous commencerons avec une représentation en deux dimensions : des fusées, des planètes, des trajectoires et des animations simples.

Nous pourrons ajouter Three.js et React Three Fiber plus tard si le temps et les performances le permettent.

PARTIE IV — LES CINQ RESPONSABILITÉS EN DÉTAIL

MEMBRE 1 — INTERFACE GÉNÉRALE ET EXPÉRIENCE DES JOUEURS

A. Sa mission

Le membre 1 est responsable de la partie visible du jeu.

Il doit faire en sorte que n'importe quel joueur puisse comprendre comment utiliser MISSION FORGE sans avoir besoin de connaître la programmation ou les sciences spatiales.

Il construit les différentes pages et organise la navigation entre elles.

B. Jeu de référence

Mars Horizon

Il doit observer particulièrement :

- L'écran d'accueil.
- Les tableaux de bord.
- La présentation des missions.
- Les informations affichées au joueur.
- Les menus.
- La progression d'une mission.

Il peut également observer Kerbal Space Program pour comprendre comment présenter les informations techniques sans surcharger l'écran.

C. Ce qu'il doit développer

Fonctionnalité	Description
Accueil	Présentation du jeu et accès aux modes
Menu principal	Accès aux différentes fonctionnalités
Choix du mode	Solo, équipe, historique ou science-fiction
Tableau de bord	Affichage du budget, de la masse et des ressources
Navigation	Passage d'une étape à une autre
Écran de mission	Affichage des informations importantes
Interface de simulation	Présentation du lancement et du déroulement du vol
Interface d'autopsie	Présentation des résultats et des explications

Il est également responsable de l'harmonisation graphique de l'application.

D. Architecture de sa partie

Il doit organiser son travail en composants.

Un composant est simplement une petite partie indépendante de l'interface.

Par exemple, le tableau de bord peut être composé de plusieurs petits éléments.

```
src/
│
├── components/
│   ├── Navbar.tsx
│   ├── Sidebar.tsx
```

```
│   ├── ResourceBar.tsx
│   ├── BudgetCard.tsx
│   ├── MissionCard.tsx
│   └── Button.tsx
│
├── pages/
│   ├── Home.tsx
│   ├── GameMode.tsx
│   ├── Dashboard.tsx
│   ├── MissionPage.tsx
│   ├── SimulationPage.tsx
│   └── ResultsPage.tsx
│
├── layouts/
│   └── MainLayout.tsx
│
└── App.tsx
```

E. Technologies et raisons de leur utilisation

Technologie	Pourquoi en a-t-il besoin ?
React	Construire les pages et les éléments de l'interface
TypeScript	Récupérer correctement les informations des autres modules
Tailwind CSS	Créer les couleurs, les espacements et les dispositions
React Router	Passer d'une page à une autre
Zustand	Afficher les informations du jeu
Framer Motion	Ajouter des animations simples

F. Comment sa partie fonctionne-t-elle avec les autres ?

Le membre 1 ne doit pas inventer les données qu'il affiche.

Exemple :

Le membre 2 construit un vaisseau avec :

- Une masse de 2 000 kg.
- Un moteur.
- Un réservoir.
- Un équipement scientifique.

Le membre 3 calcule les performances.

Le membre 1 récupère ces informations et les présente dans le tableau de bord.

Il ne doit donc pas recalculer lui-même la masse ou le carburant.

Son rôle est de présenter les informations et de permettre aux joueurs d'interagir avec le jeu.

G. Pourquoi sa partie est-elle importante ?

Même si les calculs scientifiques sont excellents, le jeu sera difficile à utiliser si les informations sont mal présentées.

Le membre 1 transforme les fonctionnalités techniques en une expérience compréhensible.

C'est lui qui permet au joueur de comprendre ce qu'il doit faire et ce qui se passe.

H. Sa feuille de route

Étape 1 : Compréhension

- Étudier Mars Horizon.
- Dessiner les différentes pages sur papier.
- Identifier les informations que chaque page doit afficher.

Étape 2 : Mise en place

- Installer React et Vite.
- Configurer TypeScript.
- Installer Tailwind CSS.
- Créer la structure des pages.

Étape 3 : Développement

- Créer l'accueil.
- Créer le menu principal.
- Créer le tableau de bord.
- Créer les écrans de mission.
- Créer l'écran de simulation.

Étape 4 : Connexion

- Connecter les pages à Zustand.
- Afficher les données du membre 2.
- Afficher les résultats du membre 3.
- Afficher les événements du membre 5.

Étape 5 : Finalisation

- Vérifier la navigation.
- Améliorer la présentation.
- Vérifier l'affichage sur différents écrans.

I. Améliorations possibles plus tard

- Animations plus avancées.
- Interface 3D.
- Mode sombre et clair.
- Personnalisation des tableaux de bord.
- Accessibilité améliorée.
- Traduction en plusieurs langues.

MEMBRE 2 — CONCEPTION ET CONSTRUCTION DU VAISSEAU SPATIAL

A. Sa mission

Le membre 2 est responsable de la conception de la mission et de la construction du vaisseau.

C'est lui qui donne aux joueurs les outils nécessaires pour choisir les composants de leur engin spatial.

Le joueur doit pouvoir sélectionner les éléments qui permettront à son vaisseau de réaliser la mission.

B. Jeux de référence

Kerbal Space Program : référence principale.

Il doit observer :

- Le constructeur de fusées.
- Le choix des moteurs.
- Les réservoirs.
- Les composants.
- La masse totale.
- L'organisation des étages.
- Les contraintes de construction.

Spaceflight Simulator : référence secondaire.

Il doit particulièrement observer la simplicité de son système de construction.

C. Ce qu'il doit développer

1. Choix de la mission

Le joueur doit pouvoir choisir un objectif.

Exemples :

- Envoyer une sonde vers Mars.
- Étudier la Lune.
- Observer un astéroïde.
- Déployer un satellite.

2. Construction du vaisseau

Le joueur doit pouvoir choisir différents composants.

Composant	Fonction
Moteur	Fournir la propulsion
Réservoir	Stocker le carburant
Panneaux solaires	Produire de l'énergie
Batterie	Stocker l'énergie
Antenne	Communiquer avec la Terre
Ordinateur de bord	Contrôler le vaisseau

Composant	Fonction
Instrument scientifique	Réaliser les observations
Structure	Maintenir les composants ensemble

3. Gestion des ressources

Le joueur doit pouvoir consulter :

- La masse totale.
- Le coût total.
- La capacité énergétique.
- La quantité de carburant.
- Les équipements installés.

4. Vérification de la construction

Le système doit vérifier les contraintes simples.

Exemples :

- Le joueur ne peut pas dépasser son budget.
- Le vaisseau doit posséder un moyen de communication adapté.
- Le vaisseau doit avoir une source d'énergie.
- Les équipements doivent être compatibles avec la mission.

D. Architecture de sa partie

```
src/
│
├── modules/
│   └── spacecraft/
│       ├── components/
│       │   ├── ComponentCatalog.tsx
│       │   ├── ComponentCard.tsx
│       │   ├── SpacecraftCanvas.tsx
│       │   └── SpacecraftSummary.tsx
│       │
│       ├── data/
│       │   └── spacecraftComponents.ts
│       │
│       ├── services/
│       │   ├── spacecraftBuilder.ts
│       │   └── spacecraftValidator.ts
│       │
│       └── types/
│           └── spacecraft.types.ts
```

Comment fonctionne cette architecture ?

- `ComponentCatalog` : affiche les composants disponibles.
- `ComponentCard` : présente chaque composant.
- `SpacecraftCanvas` : représente le vaisseau en construction.
- `SpacecraftSummary` : affiche les caractéristiques du vaisseau.
- `spacecraftComponents` : contient les caractéristiques des composants.
- `spacecraftBuilder` : gère l'ajout et le retrait des composants.
- `spacecraftValidator` : vérifie si la construction respecte les règles.

E. Technologies et raisons de leur utilisation

Technologie	Pourquoi ?
React	Construire l'interface de construction
TypeScript	Définir les caractéristiques des composants
Tailwind CSS	Organiser visuellement les composants
Zustand	Sauvegarder la construction actuelle dans l'état du jeu
SVG	Dessiner les fusées et leurs composants en 2D
React DnD ou équivalent	Permettre de déplacer les composants avec la souris

Il n'est pas nécessaire d'utiliser Three.js pour commencer.

SVG suffit pour construire un système de fusées en deux dimensions.

F. Comment sa partie fonctionne-t-elle avec les autres ?

Le membre 2 fournit au membre 3 les informations nécessaires pour effectuer la simulation.

Exemple :

Le joueur construit un vaisseau.

Le membre 2 transmet :

```
Vaisseau :
- Masse : 2 000 kg
- Carburant : 800 kg
- Moteur : Moteur A
- Puissance disponible : 500 W
- Capacité batterie : 2 000 Wh
- Antenne : Antenne A
- Instruments : Caméra scientifique
```

Ces informations sont ensuite utilisées par le membre 3.

Le membre 1 les affiche dans le tableau de bord.

Le membre 5 les utilise pour analyser les événements et les résultats.

G. Pourquoi sa partie est-elle importante ?

Sans ce module, les joueurs ne peuvent pas prendre de décisions techniques.

C'est lui qui donne naissance à la mission.

Les autres modules dépendent directement de ses données.

Un mauvais choix de composants peut entraîner un échec pendant la simulation.

H. Sa feuille de route

Étape 1 : Étude

- Étudier Kerbal Space Program.
- Identifier les composants essentiels.
- Comprendre le rôle de chaque composant.
- Définir les caractéristiques nécessaires.

Étape 2 : Création des données

- Créer un catalogue de composants.
- Définir leur coût.

- Définir leur masse.
- Définir leur consommation énergétique.
- Définir leurs capacités.

Étape 3 : Construction

- Afficher les composants.
- Permettre leur sélection.
- Permettre leur ajout et leur retrait.
- Afficher le résumé du vaisseau.

Étape 4 : Vérification

- Calculer la masse totale.
- Calculer le coût total.
- Vérifier les équipements obligatoires.
- Vérifier le respect du budget.

Étape 5 : Connexion

- Transmettre les données au moteur de simulation.
- Afficher les données dans le tableau de bord.
- Permettre la sauvegarde de la conception.

I. Améliorations possibles plus tard

- Construction entièrement libre.
- Plusieurs étages de fusées.
- Composants personnalisables.
- Compatibilité technique plus détaillée.
- Différents types de propulsion.
- Construction en 3D.
- Catalogue de composants basé sur des équipements spatiaux réels.

MEMBRE 3 — MOTEUR SCIENTIFIQUE ET SIMULATION SPATIALE

A. Sa mission

Le membre 3 est responsable de la partie scientifique du jeu.

Il doit transformer les caractéristiques du vaisseau en résultats concrets.

Son travail consiste à répondre à une question fondamentale :

Est-ce que le vaisseau construit par les joueurs peut réellement accomplir la mission prévue dans les conditions choisies par le jeu ?

Il doit également simuler le déroulement du voyage.

B. Jeux de référence

Juno: New Origins : référence principale.

Observer :

- Les calculs physiques.
- Les performances des moteurs.
- Les mouvements des engins.
- Les simulations.

Kerbal Space Program : référence secondaire.

Observer :

- Le lancement.
- La consommation de carburant.
- Les trajectoires.
- Les mouvements orbitaux.

Universe Sandbox : référence complémentaire.

Observer la représentation de la gravité et les interactions entre les corps célestes.

C. Ce qu'il doit développer

1. Calcul de la masse

Le moteur doit additionner les masses des composants.

Exemple :

Élément	Masse
Structure	400 kg
Moteur	300 kg
Réservoir vide	100 kg
Carburant	800 kg
Instruments	200 kg
Ordinateur	200 kg
Total	2 000 kg

2. Calcul du carburant

Le système doit déterminer si le carburant disponible est suffisant pour effectuer la mission.

3. Calcul de l'énergie

Il doit comparer :

- L'énergie produite.
- L'énergie stockée.
- L'énergie consommée.

Exemple :

Un vaisseau consomme 200 W et produit 150 W.

Il possède donc un déficit énergétique de 50 W tant que cette situation dure.

4. Calcul des performances

Le système doit évaluer les performances du vaisseau en fonction de ses équipements.

Exemples :

- Puissance du moteur.
- Masse totale.
- Capacité énergétique.
- Réserve de carburant.
- Capacité de communication.

5. Simulation du voyage

Le système doit représenter les grandes étapes de la mission :

- Départ.
- Propulsion.
- Voyage.
- Arrivée.
- Réalisation de l'objectif.
- Retour ou fin de mission.

6. Gestion du temps

Les joueurs doivent pouvoir observer l'évolution de la mission dans le temps.

Exemple :

```
00:00 : Début du lancement
00:30 : Phase de propulsion
02:00 : Mise en orbite
10:00 : Début du voyage
...
```

Ces temps pourront être accélérés dans le jeu pour rendre les parties plus courtes.

D. Architecture de sa partie

```
src/
│
├── modules/
│   └── simulation/
│       ├── calculations/
│       │   ├── massCalculator.ts
│       │   ├── fuelCalculator.ts
│       │   ├── energyCalculator.ts
│       │   └── propulsionCalculator.ts
│       │
│       ├── physics/
│       │   ├── gravity.ts
│       │   ├── orbit.ts
│       │   └── trajectory.ts
│       │
│       ├── engine/
│       │   ├── simulationEngine.ts
│       │   └── simulationEvents.ts
│       │
│       ├── services/
│       │   └── missionEvaluator.ts
│       │
│       └── types/
│           └── simulation.types.ts
```

Explication

- `massCalculator` : calcule la masse.
- `fuelCalculator` : calcule la consommation de carburant.
- `energyCalculator` : vérifie les ressources énergétiques.
- `propulsionCalculator` : évalue les performances du moteur.
- `gravity` : représente les effets de la gravité.
- `orbit` : calcule les mouvements orbitaux.
- `trajectory` : détermine le déplacement de l'engin.
- `simulationEngine` : organise le déroulement du voyage.
- `simulationEvents` : reçoit les événements qui se produisent.
- `missionEvaluator` : vérifie si les objectifs sont atteints.

E. Technologies et raisons de leur utilisation

Technologie	Pourquoi ?
TypeScript	Effectuer les calculs et organiser les règles scientifiques
Zustand	Récupérer les caractéristiques du vaisseau
Vitest	Vérifier que les calculs donnent les bons résultats
SVG ou Canvas	Représenter les trajectoires et les mouvements
Python + NumPy/SciPy	Facultatif, pour des calculs scientifiques plus complexes

Important : les calculs de base doivent rester en TypeScript afin de simplifier la communication avec les autres modules.

Python ne sera ajouté que si une nécessité scientifique le justifie.

F. Comment sa partie fonctionne-t-elle avec les autres ?

Le membre 3 reçoit les caractéristiques du vaisseau du membre 2.

Il reçoit également les objectifs de la mission.

Il effectue les calculs et transmet les résultats :

- Au membre 1 pour l'affichage.
- Au membre 5 pour l'analyse des événements et les explications.
- Au membre 4 pour la sauvegarde des résultats.

Exemple :

Le membre 2 construit un vaisseau.

Le membre 3 constate que sa réserve énergétique ne permet pas de faire fonctionner les instruments pendant toute la durée prévue.

Le membre 1 affiche une alerte.

Le membre 5 pourra ensuite expliquer pourquoi cette décision a provoqué une panne pendant le voyage.

G. Pourquoi sa partie est-elle importante ?

C'est le moteur du jeu.

Sans lui, MISSION FORGE ne serait qu'un constructeur de fusées avec des animations.

Grâce à lui, les décisions des joueurs auront des conséquences.

Il donne une valeur scientifique et éducative au projet.

H. Sa feuille de route

Étape 1 : Définir les règles

- Identifier les calculs nécessaires.
- Définir les unités utilisées.
- Définir les limites de la simulation.
- Déterminer les règles simplifiées du jeu.

Étape 2 : Développer les calculs simples

- Masse.
- Budget.
- Carburant.
- Énergie.
- Puissance.

Étape 3 : Développer les performances

- Évaluer les moteurs.
- Vérifier les besoins de la mission.
- Détecter les insuffisances techniques.

Étape 4 : Développer la simulation

- Organiser les étapes du voyage.
- Faire évoluer les ressources.
- Ajouter les déplacements.
- Gérer le temps.

Étape 5 : Connexion

- Recevoir les données du membre 2.
- Transmettre les résultats au membre 1.
- Transmettre les événements au membre 5.

Étape 6 : Vérification

- Tester les calculs.
- Comparer les résultats avec des exemples connus.
- Vérifier les cas de réussite et d'échec.

I. Améliorations possibles plus tard

- Modèles orbitaux plus précis.
- Gravitation à plusieurs corps.
- Différents types de propulsion.
- Simulation plus réaliste des trajectoires.
- Prise en compte de la résistance atmosphérique.

- Simulation thermique.
- Modèles de dégradation des équipements.

Il faudra documenter les simplifications utilisées pour ne pas présenter les résultats du jeu comme des prédictions exactes de la réalité.

MEMBRE 4 — MULTIJOUEUR, COMMUNICATION ET SAUVEGARDE

A. Sa mission

Le membre 4 est responsable de la collaboration entre les joueurs et de la conservation des données.

Le cahier des charges prévoit que quatre joueurs maximum puissent participer à une même mission.

Mais il y a une particularité importante :

Chaque joueur doit disposer d'une vue partielle de la mission.

Cela signifie que les quatre joueurs ne doivent pas nécessairement voir les mêmes informations.

Par exemple :

Joueur	Responsabilité dans le jeu
Joueur 1	Gestion du budget
Joueur 2	Conception du vaisseau
Joueur 3	Énergie et communication
Joueur 4	Objectifs scientifiques

Chaque joueur possède des informations et des responsabilités particulières.

Ils doivent discuter pour prendre les bonnes décisions.

B. Jeu de référence

Mars Horizon : référence principale.

Observer :

- La gestion d'une agence spatiale.
- La planification.
- La gestion des ressources.
- La coordination des différentes responsabilités.

Il faudra également étudier les systèmes de coopération d'autres jeux de gestion.

C. Ce qu'il doit développer

1. Création d'une partie

Un joueur peut créer une salle de jeu.

Le système lui attribue un code de session.

Exemple :

```
MISSION FORGE
```

```
Nouvelle partie créée

Code : MF2048

Joueurs : 1/4

En attente des autres membres...
```

2. Rejoindre une partie

Les autres joueurs utilisent le code pour rejoindre la session.

3. Attribution des rôles

Le système attribue une responsabilité à chaque joueur.

4. Synchronisation

Lorsqu'un joueur modifie une information importante, les autres joueurs doivent recevoir la modification correspondante.

5. Sauvegarde

Le système doit conserver :

- Les informations des joueurs.
- Les missions.
- Les vaisseaux.
- Les décisions.
- Les résultats.
- La progression.

6. Gestion des déconnexions

Si un joueur perd sa connexion, sa progression ne doit pas disparaître.

Le système doit prévoir la récupération de la session.

D. Architecture de sa partie

```
src/
│
├── modules/
│   └── multiplayer/
│       ├── components/
│       │   ├── CreateRoom.tsx
│       │   ├── JoinRoom.tsx
│       │   ├── PlayerList.tsx
│       │   └── PlayerRole.tsx
│       │
│       ├── services/
│       │   ├── roomService.ts
│       │   ├── playerService.ts
│       │   └── synchronizationService.ts
│       │
│       └── types/
│           └── multiplayer.types.ts
│
├── modules/
│   └── persistence/
│       ├── saveMission.ts
│       ├── loadMission.ts
│       └── missionRepository.ts
│
└── lib/
    └── supabase.ts
```

E. Technologies et raisons de leur utilisation

Technologie	Pourquoi ?
TypeScript	Organiser les données des joueurs et des sessions
React	Créer les interfaces de connexion
Supabase Auth	Gérer l'identification des joueurs
Supabase PostgreSQL	Sauvegarder les données
Supabase Realtime	Transmettre certaines modifications aux autres joueurs
Zustand	Maintenir les informations de la partie dans l'application

Socket.IO pourra être envisagé si Supabase Realtime ne répond pas à nos besoins.

Il ne faut pas utiliser les deux systèmes simultanément sans raison.

Pourquoi Supabase ?

Supabase fournit plusieurs services déjà prêts :

- Une base de données.
- Un système d'authentification.
- Des fonctions de communication en temps réel.
- Des règles permettant de contrôler l'accès aux données.

Cela évite de développer tous ces systèmes depuis zéro.

F. Comment sa partie fonctionne-t-elle avec les autres ?

Le membre 4 ne décide pas du contenu scientifique des missions.

Il permet simplement de partager et conserver les informations produites par les autres modules.

Exemple :

1. Le membre 2 modifie le vaisseau.
2. Le membre 3 vérifie les nouvelles caractéristiques.
3. Le membre 4 sauvegarde la modification.
4. Les autres joueurs reçoivent les nouvelles informations.
5. Le membre 1 actualise l'affichage.

G. Pourquoi sa partie est-elle importante ?

Le cahier des charges demande une expérience collaborative.

Sans ce module, le jeu risque de fonctionner uniquement en solo.

Il est aussi indispensable pour éviter la perte de progression et permettre aux joueurs de retrouver leur mission.

H. Sa feuille de route

Étape 1 : Préparation

- Étudier les besoins du mode multijoueur.
- Définir les informations des joueurs.
- Définir les rôles.
- Définir les règles de création des salles.

Étape 2 : Base de données

- Créer les tables des utilisateurs.
- Créer les tables des sessions.
- Créer les tables des missions.
- Créer les tables des joueurs.

Étape 3 : Création des sessions

- Créer une salle.
- Générer un code.
- Rejoindre une salle.
- Afficher les participants.

Étape 4 : Collaboration

- Attribuer les rôles.
- Partager les modifications.
- Synchroniser les données.

Étape 5 : Sauvegarde

- Enregistrer les missions.
- Charger une mission.
- Enregistrer les résultats.

Étape 6 : Tests

- Tester plusieurs joueurs.
- Tester les déconnexions.
- Tester les reconnexions.
- Vérifier que deux modifications simultanées ne détruisent pas les données.

I. Améliorations possibles plus tard

- Chat intégré.
- Invitations par lien.
- Historique des décisions.
- Gestion avancée des droits.
- Mode spectateur.
- Sauvegarde automatique.
- Reprise d'une mission après interruption.

MEMBRE 5 — MISSIONS, ÉVÉNEMENTS, RISQUES ET AUTOPSIE

A. Sa mission

Le membre 5 est responsable de la dimension narrative, pédagogique et analytique du jeu.

Son rôle est de donner un objectif aux joueurs, de créer des situations intéressantes et de leur expliquer les conséquences de leurs décisions.

Il est responsable de quatre grands éléments :

1. Les missions historiques.
2. Les scénarios de science-fiction.
3. Les événements imprévus.
4. L'autopsie de mission.

Il contribue également à l'explorateur de l'univers.

B. Jeux de référence

Kerbal Space Program : référence principale pour les événements et les situations imprévues.

Observer :

- Les accidents.
- Les difficultés rencontrées pendant les voyages.
- Les situations qui obligent à modifier une stratégie.

Universe Sandbox : référence pour l'exploration et la visualisation.

Observer :

- Les objets célestes.
- Les systèmes planétaires.
- Les informations scientifiques.

C. Ce qu'il doit développer

1. Catalogue des missions

Il doit créer différents scénarios.

Exemple :

```
MISSION : MARS EXPLORER

Destination : Mars

Objectif :
Étudier la surface de Mars.

Contraintes :
- Budget limité.
- Énergie limitée.
- Durée de mission déterminée.
- Communication avec la Terre.

Équipement nécessaire :
- Caméra scientifique.
- Antenne.
- Source d'énergie.
```

2. Missions historiques

Il doit constituer une base de missions réelles.

Chaque mission devra contenir :

- Son nom.
- Son année.
- Son objectif.
- Sa destination.
- Les équipements importants.
- Les principales étapes.

- Son résultat.
- Ses sources documentaires.

Les données historiques doivent être vérifiées auprès de sources fiables, notamment la NASA.

3. Événements imprévus

Pendant une simulation, le jeu pourra déclencher des événements.

Exemples :

Événement	Conséquence possible
Panne d'un panneau solaire	Diminution de l'énergie disponible
Perte de communication	Impossibilité de recevoir des instructions
Défaillance du moteur	Modification de la trajectoire
Surconsommation	Réduction de la durée de mission
Défaillance d'un instrument	Perte de données scientifiques

Ces événements doivent être liés aux caractéristiques du vaisseau et à la situation de la mission.

Il ne faut pas que les pannes soient simplement choisies au hasard sans logique.

4. Système de risques

Le jeu doit évaluer certains risques avant et pendant la mission.

Exemple :

Un vaisseau ne possède qu'une seule source d'énergie.

Une panne de cette source peut provoquer la perte de plusieurs équipements.

Le système doit donc considérer cette situation comme risquée.

5. Autopsie de mission

C'est l'une des fonctionnalités les plus importantes de notre projet.

Après chaque mission, le système doit expliquer les résultats.

Exemple :

```
AUTOPSIE DE MISSION

Résultat : ÉCHEC PARTIEL

OBJECTIF SCIENTIFIQUE
Non atteint.

PROBLÈME PRINCIPAL
Énergie insuffisante.

CAUSE
Les panneaux solaires ne produisaient
pas suffisamment d'énergie pour alimenter
les équipements.

CONSÉQUENCE
L'ordinateur de bord a réduit les
fonctions non prioritaires.

DÉCISION À L'ORIGINE
Choix d'une source d'énergie
insuffisante.

AMÉLIORATION
Installer une source d'énergie plus
performante ou réduire la consommation.
```

L'autopsie doit s'appuyer sur les événements réellement enregistrés pendant la simulation.

Elle ne doit pas inventer une cause après coup.

6. Explorateur de l'univers

Le membre 5 pourra également construire un catalogue des objets célestes.

Exemple :

- Terre.
- Lune.
- Mars.
- Jupiter.
- Saturne.
- Certaines exoplanètes.

Les informations peuvent provenir des sources scientifiques de la NASA et du JPL.

D. Architecture de sa partie

```
src/
│
├── modules/
│   └── mission-intelligence/
│       │
│       ├── missions/
│       │   ├── missionCatalog.ts
│       │   ├── historicalMissions.ts
│       │   └── fictionalMissions.ts
│       │
│       ├── events/
│       │   ├── eventCatalog.ts
│       │   ├── eventGenerator.ts
│       │   └── eventRules.ts
│       │
│       ├── risks/
│       │   └── riskEvaluator.ts
│       │
│       ├── autopsy/
│       │   ├── autopsyGenerator.ts
│       │   ├── causeAnalyzer.ts
│       │   └── missionTimeline.ts
│       │
│       ├── universe/
│       │   ├── celestialCatalog.ts
│       │   └── celestialData.ts
│       │
│       └── types/
│           └── mission-intelligence.types.ts
```

E. Technologies et raisons de leur utilisation

Technologie	Pourquoi ?
TypeScript	Organiser les missions et les événements
React	Présenter les missions et les résultats
Zustand	Récupérer les données de la mission en cours
Vitest	Vérifier les règles et les événements
NASA Open APIs	Récupérer certaines données scientifiques
JPL Horizons	Obtenir des informations astronomiques et des éphémérides
SVG	Représenter certaines trajectoires et visualisations simples

Le membre 5 n'a pas besoin d'utiliser une intelligence artificielle pour générer les autopsies.

Une analyse basée sur des règles claires sera plus facile à développer, à tester et à expliquer.

F. Comment sa partie fonctionne-t-elle avec les autres ?

C'est le module qui rassemble les informations produites par les autres membres.

Exemple :

1. Le membre 2 fournit les caractéristiques du vaisseau.
2. Le membre 3 effectue la simulation.
3. Le membre 3 enregistre les événements et les changements de ressources.
4. Le membre 5 analyse ces informations.
5. Le membre 1 affiche l'autopsie.
6. Le membre 4 sauvegarde le résultat.

G. Pourquoi sa partie est-elle importante ?

Sans ce module, le jeu pourrait simplement afficher une mission réussie ou échouée.

Mais le cahier des charges demande davantage : les joueurs doivent comprendre pourquoi leurs décisions ont conduit à ce résultat.

C'est donc ce module qui transforme une simulation en véritable expérience d'apprentissage.

Il donne également au jeu sa richesse narrative grâce aux missions historiques et aux scénarios imaginaires.

H. Sa feuille de route

Étape 1 : Recherche

- Étudier les missions spatiales réelles.
- Identifier les informations scientifiques disponibles.
- Définir les types de scénarios.

Étape 2 : Création des missions

- Créer les missions historiques.
- Créer les missions fictives.
- Définir leurs objectifs et contraintes.

Étape 3 : Événements

- Créer un catalogue de pannes.
- Définir leurs conditions de déclenchement.
- Définir leurs conséquences.

Étape 4 : Risques

- Définir les règles d'évaluation des risques.
- Relier les risques aux caractéristiques du vaisseau.
- Prévoir les conséquences possibles.

Étape 5 : Autopsie

- Récupérer les événements.
- Identifier les causes.
- Relier les causes aux décisions.

- Générer un rapport compréhensible.

Étape 6 : Explorateur

- Constituer un catalogue d'objets célestes.
- Ajouter les informations scientifiques.
- Ajouter les sources.

Étape 7 : Connexion

- Récupérer les résultats du membre 3.
- Afficher les analyses dans l'interface du membre 1.
- Sauvegarder les rapports avec le membre 4.

I. Améliorations possibles plus tard

- Davantage de missions historiques.
- Scénarios générés dynamiquement.
- Événements plus complexes.
- Analyse détaillée des décisions.
- Système de notation scientifique.
- Comparaison de plusieurs stratégies.
- Explorateur 3D.
- Explications pédagogiques plus avancées.

PARTIE V — COMMENT LES CINQ ARCHITECTURES DOIVENT S'ASSEMBLER

C'est la partie la plus importante pour éviter les problèmes d'intégration.

Chaque membre peut avoir une architecture interne différente, mais tous doivent respecter une organisation commune.

5.1. Architecture finale du projet

Voici l'organisation générale recommandée du dépôt GitHub :

```
MISSION-FORGE/
│
├── public/
│   ├── images/
│   └── icons/
│
├── src/
│   │
│   ├── app/
│   │   ├── App.tsx
│   │   ├── router.tsx
│   │   └── providers.tsx
│   │
│   ├── components/
│   │   ├── Navbar.tsx
│   │   ├── Sidebar.tsx
│   │   ├── Button.tsx
│   │   └── ResourceBar.tsx
│   │
│   ├── modules/
│   │   ├── spacecraft/
│   │   │   └── ...
│   │   │
```

```
│   │   ├── simulation/
│   │   │   └── ...
│   │   │
│   │   ├── multiplayer/
│   │   │   └── ...
│   │   │
│   │   └── mission-intelligence/
│   │       └── ...
│   │
│   ├── pages/
│   │   ├── Home.tsx
│   │   ├── Dashboard.tsx
│   │   ├── GameMode.tsx
│   │   └── Results.tsx
│   │
│   ├── store/
│   │   ├── gameStore.ts
│   │   └── missionStore.ts
│   │
│   ├── shared/
│   │   ├── types/
│   │   │   ├── mission.types.ts
│   │   │   ├── spacecraft.types.ts
│   │   │   ├── simulation.types.ts
│   │   │   ├── player.types.ts
│   │   │   └── event.types.ts
│   │   │
│   │   ├── constants/
│   │   └── utils/
│   │
│   ├── services/
│   │   ├── nasaApi.ts
│   │   └── database.ts
│   │
│   └── main.tsx
│
├── tests/
│   ├── spacecraft/
│   ├── simulation/
│   ├── multiplayer/
│   └── mission-intelligence/
│
├── docs/
│   ├── ARCHITECTURE.md
│   ├── API-CONTRACTS.md
│   ├── GAME-RULES.md
│   └── CONTRIBUTING.md
│
├── package.json
├── tsconfig.json
├── vite.config.ts
└── README.md
```

Les trois points `...` représentent les fichiers internes que chaque membre développera.

5.2. Les règles communes de communication

Pour éviter que chacun utilise des noms différents pour désigner la même information, nous devons définir des structures communes.

Ces structures sont appelées des types.

Par exemple, tous les membres doivent comprendre ce qu'est une mission.

Voici une première définition commune :

```
export interface Mission {
  id: string;
  name: string;
  destination: string;
  objective: string;
  budget: number;
  duration: number;
  status: "design" | "ready" | "running" | "completed" | "failed";
```

```
}
```

Cela signifie que chaque mission possède obligatoirement :

- Un identifiant.
- Un nom.
- Une destination.
- Un objectif.
- Un budget.
- Une durée.
- Un état.

Autre exemple :

```
export interface Spacecraft {
  id: string;
  name: string;
  components: SpacecraftComponent[];
  totalMass: number;
  totalCost: number;
  fuelCapacity: number;
  energyCapacity: number;
}
```

Le membre 2 produit ces données.

Le membre 3 les utilise.

Le membre 1 les affiche.

Le membre 4 les sauvegarde.

Le membre 5 les analyse.

C'est ainsi que les cinq architectures deviennent une seule architecture cohérente.

5.3. Les informations que chaque membre doit fournir

Membre	Informations qu'il doit fournir
1	Interfaces et pages permettant d'afficher les données
2	Caractéristiques complètes du vaisseau
3	Résultats des calculs et de la simulation
4	Informations des joueurs, sessions et sauvegardes
5	Missions, événements, risques et rapports d'autopsie

5.4. Le fonctionnement général des données

Prenons un exemple concret.

Un joueur souhaite envoyer une sonde sur Mars.

Étape 1 : membre 5

Le jeu propose la mission Mars Explorer avec ses objectifs et contraintes.

Étape 2 : membre 1

Le joueur découvre la mission et accède à l'écran de conception.

Étape 3 : membre 2

Le joueur choisit les équipements et construit son vaisseau.

Étape 4 : membre 3

Le système calcule si le vaisseau est capable de réaliser la mission.

Étape 5 : membre 4

La conception est partagée avec les autres joueurs et sauvegardée.

Étape 6 : membre 3

La simulation commence et produit des résultats.

Étape 7 : membre 5

Le système analyse les événements et détermine les causes de réussite ou d'échec.

Étape 8 : membre 1

Les joueurs découvrent le rapport final.

PARTIE VI — ORDRE DE DÉVELOPPEMENT ET DÉPENDANCES

6.1. Qui doit commencer avant qui ?

Les membres peuvent travailler en parallèle, mais certaines informations doivent être définies avant le développement complet.

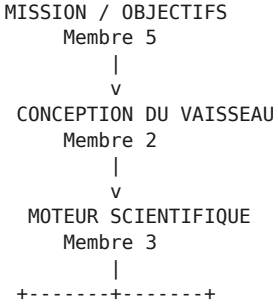
Voici l'ordre logique :

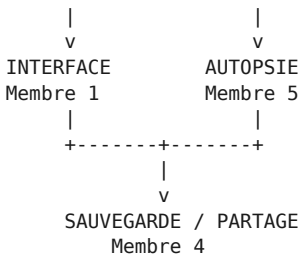
Priorité	Travail	Responsable
1	Définir les règles communes et les données	Tous
2	Créer les interfaces de base	Membre 1
3	Définir les composants du vaisseau	Membre 2
4	Définir les règles scientifiques	Membre 3
5	Définir les données des joueurs	Membre 4
6	Créer les premières missions et événements	Membre 5
7	Connecter les modules	Tous
8	Tester le jeu complet	Tous

Il ne faut pas attendre qu'un membre termine entièrement son travail avant que les autres commencent.

Par exemple, le membre 3 peut commencer à développer ses calculs avec des données fictives avant que le membre 2 ait terminé son constructeur.

6.2. Les dépendances entre les modules





Le membre 4 peut commencer son travail dès le début en utilisant des données d'exemple. Il n'est pas nécessaire qu'il attende les autres.

6.3. Les éléments qui doivent être validés ensemble

Avant de commencer à développer, toute l'équipe doit se mettre d'accord sur :

1. Les composants disponibles.
2. Les caractéristiques de chaque composant.
3. Les unités utilisées.
4. Les ressources gérées.
5. Les règles de simulation.
6. Les conditions de réussite.
7. Les conditions d'échec.
8. Les informations conservées dans l'historique.

Cette étape évitera beaucoup de conflits.

PARTIE VII — FEUILLE DE ROUTE GÉNÉRALE DE TERA VISION

Nous allons organiser le travail en quatre grandes phases.

PHASE 1 — Préparation et architecture

Objectif : préparer le terrain.

Travaux :

- Créer le dépôt GitHub.
- Configurer React, Vite et TypeScript.
- Installer les dépendances communes.
- Définir les types partagés.
- Définir les règles du jeu.
- Répartir les responsabilités.
- Créer les premières maquettes.

Résultat attendu : tous les membres possèdent une base commune.

PHASE 2 — Développement du prototype

Objectif : obtenir une première version jouable.

Fonctionnalités prioritaires :

- Choisir une mission.
- Construire un vaisseau.
- Calculer la masse.
- Calculer le budget.
- Vérifier l'énergie.
- Lancer une simulation simplifiée.
- Afficher le résultat.

À cette étape, le mode multijoueur peut encore être simulé localement.

Résultat attendu : une mission complète peut être jouée en solo.

PHASE 3 — Intégration des fonctionnalités

Objectif : transformer le prototype en véritable jeu.

Ajouter :

- Les événements imprévus.
- Les risques.
- L'autopsie.
- Le mode multijoueur.
- La sauvegarde.
- Les missions historiques.
- Les missions de science-fiction.
- L'explorateur de l'univers.

Résultat attendu : les fonctionnalités essentielles du cahier des charges sont présentes.

PHASE 4 — Finalisation

Objectif : rendre le jeu présentable et fiable.

Travaux :

- Corriger les erreurs.
- Tester les calculs.
- Vérifier la synchronisation.
- Améliorer l'interface.
- Vérifier les sources scientifiques.
- Déployer le jeu.
- Préparer une démonstration.

Résultat attendu : un jeu accessible depuis un simple lien et capable de démontrer sa valeur pédagogique.

PARTIE VIII — CE QUE NOUS DEVONS ABSOLUMENT RÉUSSIR

Le cahier des charges contient beaucoup d'idées, mais nous devons distinguer les éléments indispensables des améliorations.

8.1. Fonctionnalités indispensables

Fonctionnalité	Priorité
Choisir une mission	Très élevée
Construire un vaisseau	Très élevée
Gérer le budget et les ressources	Très élevée
Vérifier la faisabilité	Très élevée
Simuler une mission	Très élevée
Déclencher des imprévus	Très élevée
Produire une autopsie explicative	Très élevée
Interface compréhensible	Très élevée

8.2. Fonctionnalités importantes à ajouter ensuite

Fonctionnalité	Priorité
Mode multijoueur à quatre	Élevée
Vues différentes selon les rôles	Élevée
Défense du projet devant un comité	Élevée
Missions historiques	Élevée
Missions de science-fiction	Élevée
Explorateur de l'univers	Moyenne

8.3. Fonctionnalités avancées

Fonctionnalité	Priorité
Simulation orbitale très précise	Faible au départ
Visualisation 3D complète	Faible au départ
Intelligence artificielle	Faible au départ
Construction physique très détaillée	Faible au départ
Simulation de tous les systèmes d'un véritable vaisseau	Faible au départ

Il vaut mieux livrer un jeu simple, cohérent et fonctionnel qu'un jeu très ambitieux dont les principales fonctionnalités ne fonctionnent pas.

PARTIE IX — ORGANISATION DU TRAVAIL SUR GITHUB

9.1. Organisation des branches

Nous recommandons les branches suivantes :

```
main
│
├── develop
│
├── feature/interface
│
├── feature/spacecraft
│
├── feature/simulation
│
├── feature/multiplayer
│
└── feature/mission-intelligence
```

Explication :

- `main` : version stable du projet.
- `develop` : version sur laquelle les différentes parties sont assemblées.
- Les branches `feature` : branches de travail individuelles.

Chaque membre développe dans sa propre branche.

Lorsqu'une fonctionnalité est terminée et testée, il propose de l'ajouter à `develop` par une Pull Request.

Une Pull Request est une demande de validation avant d'intégrer son code dans la version commune.

9.2. Règles de collaboration

1. Ne jamais modifier directement la branche `main`.
2. Ne jamais modifier les fichiers d'un autre membre sans coordination.
3. Prévenir l'équipe lorsqu'un fichier commun doit être modifié.
4. Tester son code avant de le partager.
5. Faire des commits avec des messages compréhensibles.
6. Ne jamais intégrer du code non fonctionnel dans la version commune.
7. Documenter les fonctions importantes.
8. Signaler rapidement les problèmes de compatibilité.

9.3. Exemple de message de commit

```
feat(spacecraft): add spacecraft component catalog
```

Cela signifie que le membre a ajouté le catalogue des composants du vaisseau.

Autres exemples :

```
feat(simulation): add fuel calculation
```

```
feat(multiplayer): create multiplayer room
```

```
feat(interface): add mission dashboard
```

```
feat(autopsy): add mission failure analysis
```

```
fix(simulation): correct energy calculation
```

PARTIE X — COMMENT SAVOIR SI CHAQUE MEMBRE A TERMINÉ SON TRAVAIL ?

Chaque responsable doit avoir des résultats vérifiables.

Membre	Son travail est considéré comme fonctionnel lorsque...
1	Le joueur peut naviguer entre les pages et consulter les informations
2	Le joueur peut construire un vaisseau et consulter ses caractéristiques
3	Le système peut calculer les performances et simuler une mission
4	Les joueurs peuvent rejoindre une partie et conserver leur progression
5	Le jeu peut proposer des missions, déclencher des événements et expliquer les résultats

Il ne suffit pas que le code existe.

Il faut pouvoir démontrer que chaque fonctionnalité fonctionne.

PARTIE XI — CONSEILS POUR LES MEMBRES DE L'ÉQUIPE

Chaque membre doit respecter une méthode de travail commune.

1. Avant de coder

- Comprendre le besoin.
- Étudier le jeu de référence.
- Lire l'architecture générale.
- Identifier les données nécessaires.
- Définir les échanges avec les autres membres.

2. Pendant le développement

- Commencer par une version simple.
- Tester chaque fonctionnalité.
- Utiliser TypeScript.
- Respecter les types partagés.
- Documenter le fonctionnement.

3. Après le développement

- Tester le module.
- Vérifier qu'il fonctionne avec les autres.
- Présenter le résultat à l'équipe.
- Corriger les problèmes.
- Améliorer progressivement.

4. Une règle importante

Aucun membre ne doit développer une fonctionnalité simplement parce qu'elle lui semble intéressante.

Chaque fonctionnalité doit contribuer directement à l'objectif du jeu.

CONCLUSION GÉNÉRALE

MISSION FORGE doit être pensé comme un système composé de cinq grandes parties complémentaires.

Le membre 1 rend le jeu accessible et compréhensible.

Le membre 2 permet aux joueurs de concevoir leurs engins spatiaux.

Le membre 3 donne une réalité scientifique aux décisions prises.

Le membre 4 permet aux joueurs de collaborer et conserve leurs données.

Le membre 5 donne du sens aux missions, crée les événements et explique les résultats.

Aucun de ces modules ne doit fonctionner indépendamment des autres.

Notre réussite dépendra de notre capacité à construire cinq parties qui communiquent correctement et qui forment une expérience unique.

Notre objectif final n'est pas seulement de permettre aux joueurs de lancer une fusée. C'est de leur permettre de comprendre pourquoi une mission spatiale réussit, échoue ou doit être adaptée.

TERA VISION — MISSION FORGE
Build Your Way to Space.